

Physics Informed Machine Learning for Parameter Estimation in Rigid Multi-Quadrotor Transport System

Miqdad Abdurrahman, Anggera Bayuwindra, Hilton Tnunay
Sekolah Teknik Elektro dan Informatika (STEI)
Institut Teknologi Bandung (ITB)
Bandung, Indonesia

Email: 23224011@mahasiswa.itb.ac.id, bayuwindra@itb.ac.id, hilton.tnunay@itb.ac.id

Abstract—Cooperative multi-quadrotor systems rigidly attached to a common frame excel at heavy-lift transportation. However, unmodeled payload can introduce continuous disturbance torques, degrading controller performance and causing severe lateral drift. This paper introduces an in-the-loop Physics-Informed Neural Network (PINN) parameter estimation framework to dynamically identify unmodeled off-center payload. Unlike classical batch least-squares or Concurrent Learning frameworks that require persistent excitation or mathematically full-rank history stacks, the proposed PINN utilizes continuous gradient descent over a short-term receding horizon. By utilizing translational and rotational physics residuals using Euler-Lagrange and Euler-Poincaré equations directly on the Special Orthogonal group, $SO(3)$, the estimated parameters continuously update the control allocation matrix, enabling adaptive thrust reallocation. The Simulations confirm that the PINN estimator rapidly converges, arrests transient lateral drift, and successfully restores trajectory tracking stability.¹

Index Terms—Multi-Agent Systems, Physics-Informed Neural Networks, Cooperative Payload Transport, Parameter Estimation, Unmanned Aerial Vehicles.

I. INTRODUCTION

REFINE INTRODUCTION LATER

A. Motivation and Related Works

Cooperative multi-quadrotor systems provide scalable, high-capacity solutions for payload transportation [1], [2]. While cable-suspended payloads [3], [4] are common, rigidly attaching quadrotors to a common frame allows the distributed system to be controlled as a single composite rigid body [2]. However, this geometric simplification assumes a perfectly symmetrical mass distribution [5], [6]. In practice, uncertain payload Centers of Gravity (CoG) introduce unmodeled disturbance torques, causing severe lateral drift that may leads to crash [7], [8].

Recent mitigations include structural hardware optimization—such as physically canting rotors to artificially increase yaw authority [2]—and algorithmic adaptation. While digital twin (DT) model predictive control [8] and Concurrent Learning [7] effectively adapt to unknown parameters, they

often suffer from computational latency or require persistent excitation during vulnerable transient phases.

To mitigate these unmodeled disturbances, recent literature has explored online parameter estimation and advanced control architectures. Classical approaches, such as the batch and recursive least-squares algorithms utilized in [5], require strict persistent excitation to prevent regressor matrix singularities. To overcome the need for continuous excitation, Concurrent Learning (CL) frameworks [7] have been introduced. While CL successfully adapts to unknown parameters, convergence is mathematically guaranteed only when the recorded history stack achieves full column rank. This strict linear independence requirement leaves the multi-agent system (MAS) highly vulnerable to lateral drift during the initial transient flight phase. Alternatively, computationally intensive frameworks leveraging Model Predictive Control (MPC) synchronized with Digital Twins (DT) [8] have been proposed to estimate the state of irregular payloads. However, this architecture inherently introduces network and computational latency between the physical controller and the DT simulator, degrading the real-time responsiveness required for aggressive stabilization.

The primary objective of this research is to develop an in-the-loop Physics-Informed Neural Network (PINN) estimator for cooperative, rigid multi-quadrotor payload transportation systems. Specifically, this work aims to mitigate the disturbance caused from an unmodeled off-center static payload. To bridge the gap between the PINN parameter estimator and the multi-quadrotor control system, the primary contributions of this work are:

- 1) a PINN estimator that dynamically identifies the unknown payload mass and static payload shift by utilizing a short-term receding horizon;
- 2) embedding energy-based loss function based on Euler-Lagrange and Euler-Poincaré equations on the $SO(3)$;
- 3) integration of the PINN with a controller based on [5] to continuously updated offset parameters into the control allocation matrix to immediately restore the tracking stability.

¹The source code for the PINN estimator and simulation environment is available at: https://github.com/miqdude/pinn_invese_estimator_conf_repo

II. PRELIMINARIES

A. Physics-Informed Parameter Estimation

Physics-Informed Neural Networks (PINNs) represent a paradigm shift in scientific machine learning by seamlessly embedding known physical laws into the deep learning optimization process. Unlike purely data-driven models that rely exclusively on large empirical datasets to learn system behaviors implicitly, PINNs utilize governing Partial Differential Equations (PDEs) or Ordinary Differential Equations (ODEs) as explicit regularizers within their loss function [9], [10].

The network typically consists of fully connected Artificial Neural Network (ANN) layers parameterized by weights and biases θ [9]. To illustrate the fundamental mechanism, consider a general dynamical system governed by an ODE of the form $\mathcal{F}(u, \dot{u}, \ddot{u}, t) = 0$. The PINN acts as a surrogate model, taking the independent variable time t as its sole input and outputting a predicted system state $\hat{u}(t)$:

$$\hat{u}(t) = \mathcal{N}_\theta(t). \quad (1)$$

During training, the network minimizes a composite loss function $\mathcal{L}_{total}$ that balances empirical observation with physical consistency. The data loss ($\mathcal{L}_{data}$) anchors the network to reality by minimizing the Mean Squared Error (MSE) between the network's predictions $\hat{u}(t)$ measured state u_{true} multiply with a scalar hyperparameter λ_1 at specific time steps t_i :

$$\mathcal{L}_{data} = \lambda_1 \frac{1}{N} \sum_{i=1}^N \|\hat{u}(t_i) - u_{true}(t_i)\|^2. \quad (2)$$

Simultaneously, the network evaluates the physical validity of its predictions. Rather than relying on numerical approximation, PINNs leverage Automatic Differentiation (Autograd) to compute exact analytical derivatives of the output with respect to the input (e.g., $\hat{u} = \frac{\partial \mathcal{N}_\theta}{\partial t}$). These derivatives are substituted back into the governing equation to form a physics residual $R(t) = \mathcal{F}(\hat{u}, \hat{\dot{u}}, \hat{\ddot{u}}, t)$. The physics loss ($\mathcal{L}_{physics}$) penalizes any deviation from zero across M collocation points:

$$\mathcal{L}_{physics} = \lambda_2 \frac{1}{M} \sum_{j=1}^M \|R(t_j)\|^2. \quad (3)$$

The total loss is calculated as a weighted sum of these objectives, $\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{data} + \lambda_2 \mathcal{L}_{physics}$. By minimizing this composite function, the network learns to predict states that not only match the data but also strictly obey the underlying physical mechanics of the system [10].

III. SYSTEM DESCRIPTION

A. System Modeling

The system or we called it frame comprises of four 1.5 kg quadrotors rigidly attached at a distance of XX meters from the geometric center of a 0.8 kg board (dimensions XX), as illustrated in Fig. XXX. We modelled each quadrotor as ideal actuator where the thrusts command are applied directly to the simulator engine. We define an inertial world frame $\mathcal{W}$

with an upward-pointing Z-axis, and a primary body frame $\mathcal{B}$ located at the board's center. The axes of $\mathcal{B}$ align perfectly with $\mathcal{W}$ when the system is level and unrotated. Each quadrotor maintains its own local coordinate frame, which remains strictly parallel to $\mathcal{B}$ and is offset purely by a spatial translation in the XY plane. The overall system's position in $\mathcal{W}$ is denoted by $\mathbf{r} = [x, y, z]^T$, and its orientation is parameterized by Euler angles $\eta = [\phi, \theta, \psi]^T$.

The MAS employs a centralized control architecture derived from [5]. The composite rigid body is commanded via a unified control wrench $\boldsymbol{\tau}$, defined as:

$$\boldsymbol{\tau} = [F_B, M_{xB}, M_{yB}, M_{zB}]^T, \quad (4)$$

where F_B represents the total collective thrust acting along the frame's z -axis, while M_{xB} , M_{yB} , and M_{zB} denote the respective desired roll, pitch, and yaw control moments applied about the frame which relates to individual quadrotor thrust inputs $\mathbf{u} = [F_1, F_2, F_3, F_4]^T$ via the allocation matrix $A \in \mathbb{R}^{4 \times 4}$:

$$\boldsymbol{\tau} = A\mathbf{u}, \quad (5)$$

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ y_1 & y_2 & y_3 & y_4 \\ -x_1 & -x_2 & -x_3 & -x_4 \\ c_1 & c_2 & c_3 & c_4 \end{bmatrix}. \quad (6)$$

The rows of A denote total thrust, roll moment, pitch moment, and yaw reactive torque, respectively.

B. Dynamic Thrust Allocation

A payload shifts from the body central frame, altering the effective moment arms. Let $\hat{\mathbf{d}}_{\text{cog}} = [\hat{x}_{\text{cog}}, \hat{y}_{\text{cog}}]^T$ denote the estimated offset. Redefining the lever arms as $\Delta x_i = x_i - \hat{x}_{\text{cog}}$ and $\Delta y_i = y_i - \hat{y}_{\text{cog}}$, the allocation matrix adapts dynamically:

$$A_{\text{final}} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ \Delta y_1 & \Delta y_2 & \Delta y_3 & \Delta y_4 \\ -\Delta x_1 & -\Delta x_2 & -\Delta x_3 & -\Delta x_4 \\ c_1 & c_2 & c_3 & c_4 \end{bmatrix}. \quad (7)$$

Control inputs are computed via the Moore-Penrose pseudo-inverse to handle transient rank deficiencies:

$$\mathbf{u} = A_{\text{final}}^+ \boldsymbol{\tau}. \quad (8)$$

C. Centralized Control Design

Based on [5], the controller separates position and attitude tracking. Position and velocity errors define commanded acceleration $\ddot{\mathbf{r}}_{cmd}$:

$$\mathbf{e}_p = \mathbf{r}_{des} - \mathbf{r}, \quad \mathbf{e}_v = \dot{\mathbf{r}}_{des} - \dot{\mathbf{r}}, \quad (9)$$

$$\ddot{\mathbf{r}}_{cmd} = K_p \mathbf{e}_p + K_d \mathbf{e}_v + \ddot{\mathbf{r}}_{ff}. \quad (10)$$

Desired roll (ϕ^{des}), pitch (θ^{des}), and total thrust (F_B^{des}) are extracted via small-angle approximation:

$$\begin{aligned}
\phi^{des} &= \frac{1}{g}(\ddot{r}_x^{des} \sin \psi_T - \ddot{r}_y^{des} \cos \psi_T), \\
\theta^{des} &= \frac{1}{g}(\ddot{r}_y^{des} \cos \psi_T + \ddot{r}_x^{des} \sin \psi_T), \\
F_B^{des} &= m(\ddot{r}_z^{des} + g).
\end{aligned} \tag{11}$$

Inner-loop desired moments $\mathbf{M}_B^{des}$ regulate orientation against angular velocities (p, q, r) :

$$\begin{aligned}
M_{xB}^{des} &= k_{p,\phi}(\phi^{des} - \phi) + k_{d,\phi}(p^{des} - p), \\
M_{yB}^{des} &= k_{p,\theta}(\theta^{des} - \theta) + k_{d,\theta}(q^{des} - q), \\
M_{zB}^{des} &= k_{p,\psi}(\psi^{des} - \psi) + k_{d,\psi}(r^{des} - r).
\end{aligned} \tag{12}$$

IV. PROPOSED PINN-BASED ESTIMATOR DESIGN

A. PINN Architecture

To address the 6-DOF rigid-body dynamics, our PINN framework utilizes continuous time t as the sole independent variable to solve the time-dependent ODEs and the output is a 12 states consists of position, linear velocity, attitude, and angular velocity [10]–[12]. The PINN design used in this research is described in Table. I.

TABLE I
PROPOSED FULLY CONNECTED LAYER PINN ARCHITECTURE SPECIFICATIONS

Layer	Number of Neurons	Activation Function
Input	1 (Continuous time t)	None
Hidden 1	64	tanh
Hidden 2	64	tanh
Output	12 (Kinematic state vector)	Linear

B. Loss Function Formulation

The neural network acts as a surrogate model mapping continuous time t to the predicted 12 states of the system. We define the complete state vector $\mathbf{s}$, comprising the position, linear velocity, attitude, and angular velocity:

$$\mathbf{s} = [\mathbf{r}^T, \dot{\mathbf{r}}^T, \boldsymbol{\eta}^T, \boldsymbol{\omega}^T]^T. \tag{13}$$

To train the network, we formulate a composite loss function $\mathcal{L}_{total}$ [12], [13] that simultaneously penalizes empirical data deviations and violations of rigid-body dynamics. This objective function is defined as a weighted sum of the data-driven observation loss ($\mathcal{L}_{data}$), the translational physics residual ($\mathcal{L}_{trans}$), and the rotational physics residual ($\mathcal{L}_{rot}$):

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{data} + \lambda_2 \mathcal{L}_{trans} + \lambda_3 \mathcal{L}_{rot}, \tag{14}$$

where the hyperparameters λ_1, λ_2 , and λ_3 are constants to balance the contributions of each loss term.

The data loss ensures the network's predictions ($\hat{\mathbf{s}}$) anchor to the physical reality measured by the onboard sensors. It is computed as the Mean Squared Error (MSE) over a temporal window of N collocation points:

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{s}}_i - \mathbf{s}_{true,i}\|^2. \tag{15}$$

To enforce the translational physical constraints, we formulate a residual loss based on the Euler-Lagrange equations [14]. Let $L = T_{trans} - V$ denote the system Lagrangian, where T_{trans} is the kinetic energy and V is the potential energy. The translational dynamics must satisfy:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \mathbf{v}} \right) - \frac{\partial L}{\partial \mathbf{r}} = \mathbf{F}_{ext}. \tag{16}$$

Evaluating the kinetic inertial term yields $m\dot{\mathbf{v}}$. Rather than analytically deriving the potential gradient, we leverage PyTorch's Automatic Differentiation (Autograd) engine to dynamically compute $\frac{\partial V}{\partial \mathbf{r}}$ during the forward pass. The resulting translational physics loss minimizes the residual of these forces across the temporal window:

$$\mathcal{L}_{trans} = \frac{1}{N} \sum_{i=1}^N \left\| m\dot{\mathbf{v}}_i + \frac{\partial V}{\partial \mathbf{r}_i} - \mathbf{F}_{ext,i} \right\|^2. \tag{17}$$

While translational dynamics are straightforward, evaluating rotational dynamics using minimal coordinates (e.g., Euler angles) introduces mathematical singularities, commonly known as gimbal lock, at extreme pitch or roll angles. To ensure global mathematical validity, we evaluate the rotational dynamics directly on the Special Orthogonal group, $SO(3)$. The rotational kinetic energy of the rigid multi-agent system is defined as:

$$T_{rot} = \frac{1}{2} \boldsymbol{\omega}^T \mathbf{J} \boldsymbol{\omega}, \tag{18}$$

where $\mathbf{J} \in \mathbb{R}^{3 \times 3}$ is the estimated inertia tensor. By applying the Euler-Poincaré equations of motion for rigid body we get:

$$\frac{d}{dt} \left(\frac{\partial T_{rot}}{\partial \boldsymbol{\omega}} \right) + \boldsymbol{\omega} \times \frac{\partial T_{rot}}{\partial \boldsymbol{\omega}} = \boldsymbol{\tau}_{ext}. \tag{19}$$

The partial derivative $\frac{\partial T_{rot}}{\partial \boldsymbol{\omega}} = \mathbf{J} \boldsymbol{\omega}$ represents the system's angular momentum, Eq. (19) resolves into the Newton-Euler formulation for rigid body rotation. In our cooperative framework, the total external torque $\boldsymbol{\tau}_{ext}$ comprises the known applied control wrench $\boldsymbol{\tau}_{app}$ and the unknown disturbance torque $\boldsymbol{\tau}_{dist}$ generated by the payload's CoG offset. The rotational physics residual evaluated during the forward pass is thus formulated as:

$$\mathcal{R}_{rot} = \left(\hat{\mathbf{J}} \dot{\hat{\boldsymbol{\omega}}} + \hat{\boldsymbol{\omega}} \times (\hat{\mathbf{J}} \hat{\boldsymbol{\omega}}) \right) - (\boldsymbol{\tau}_{app} - \boldsymbol{\tau}_{dist}). \tag{20}$$

Finally, the rotational loss penalizes the mean squared error of this residual, forcing the neural network to implicitly map the physical discrepancies to the true disturbance parameters:

$$\mathcal{L}_{rot} = \frac{1}{N} \sum_{i=1}^N \|\mathcal{R}_{rot,i}\|^2. \tag{21}$$

C. Simulation Design

To eliminate computational latency anomalies and isolate the convergence behavior of the online-learning PINN estimator, a synchronous, single-threaded lockstep execution architecture was utilized for this proof of concept. This guarantees temporal alignment between the plant state and the automatic differentiation tracking within the physics loss. Transitioning these mechanics to an asynchronous, dual-rate Software-in-the-Loop (SIL) framework to handle real-world computational lag is preserved for future work.

In Algorithm. 1, the PINN needs to estimate XY offsets $\hat{d}_x, \hat{d}_y$, estimated mass $\hat{m}$. For estimating the new system inertia $\hat{\mathbf{J}}$ we used manual calculation from $\hat{d}_x, \hat{d}_y$ and $\hat{m}$ with the initial inertia without payload using the parallel axes theorem cite here.

Algorithm 1 Synchronous Online Parameter Estimation Simulation

Require: Simulation duration T_{max} , Physics time step Δt
Require: PINN update interval N steps
Require: PINN optimization steps S_{steps}
Require: Replay buffer $\mathcal{B}$ with capacity W
Require: Initial parameter guesses: $\hat{d}_x, \hat{d}_y, \hat{\mathbf{J}}, \hat{m}$

- 1: Init quadrotor physical states $\mathbf{x}_0$
- 2: Init PINN $\mathcal{N}_\theta$ with random weights θ
- 3: Simulation time $t \leftarrow 0$
- 4: Step counter $k \leftarrow 0$
- 5: **while** $t \leq T_{max}$ **do**
- 6: Physics engine update state at t
- 7: Compute target states $\mathbf{p}_d, \mathbf{v}_d, \mathbf{a}_d, \dot{\psi}_d, \dot{\psi}_d$ at t
- 8: Read current state $\mathbf{x}_k = (\mathbf{p}, \mathbf{v}, \mathbf{R}, \boldsymbol{\omega})$
- 9: Append $\mathbf{x}_k$ and applied rotor thrusts $\mathbf{T}_k$ to buffer $\mathcal{B}$
- 10: **if** $k \pmod N == 0$ **and** $|\mathcal{B}| == W$ **then**
- 11: Compute composite physics loss $\mathcal{L}_{total}$ over $\mathcal{B}$
- 12: Optimize θ via backpropagation for S_{steps}
- 13: Extract updated parameters $\hat{d}_x, \hat{d}_y, \hat{m}$ from $\mathcal{N}_\theta$
- 14: Calculate $\hat{\mathbf{J}}$
- 15: **end if**
- 16: Calculate desired control wrench $\mathbf{F}_{des}, \mathbf{M}_{des}$
- 17: Construct control allocation matrix $\mathbf{A}_{final}$ using $\hat{d}_x, \hat{d}_y$
- 18: Compute thrust commands $\mathbf{u} \leftarrow \mathbf{A}_{final}^\dagger [\mathbf{F}_{des}^T, \mathbf{M}_{des}^T]^T$
- 19: Apply saturated commands $\mathbf{u}$ to physics engine
- 20: $t \leftarrow t + \Delta t$
- 21: $k \leftarrow k + 1$
- 22: **end while**

V. VALIDATION AND RESULTS

A. Simulation Environment and Setup

The simulation was done using a laptop with 3050 series 6 GB of VRAM GPU and Cuda library support. The optimizer used was Adam cite here. Table. II contains the independent variables used in Algorithm. 1.

TABLE II
SYSTEM AND SIMULATION PARAMETERS

Parameter	Symbol	Value
Simulation Time-Step	Δt	1/240 s
Gravitational Acceleration	g	9.81 m/s ²
Simulation Duration	t_{sim}	90 s
Base Frame Mass	m_f	6.8 kg
Quadrotor Mass	-	1.5 kg
Payload Mass	m_p	1.0 kg
Payload Volume	V_p	$125 \times 10^{-6} \text{ m}^3$
Total System Mass	m_{tot}	7.8 kg
Base System Inertia on X axis	J_{xx}	1.713
Base System Inertia on Y axis	J_{yy}	1.713
Base System Inertia on Z axis	J_{zz}	3.413
Max Thrust per Drone	T_{max}	50 N
Unknown Payload Offset	$d_{x,cog}, d_{y,cog}$	(-0.15, 0.15) m
PINN Window Size	W	50 steps
Optimization Frequency	N_{opt}	5 steps
Gradient Steps per Cycle	S_{grad}	3 steps

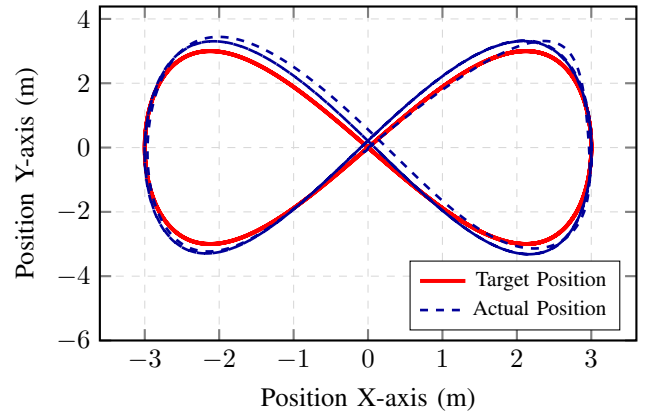


Fig. 1. The body is able to follow the desired trajectory without crashing at 1st episode

B. Parameter Convergence and Estimation

Bahas ablation studynya disini

Operating over the receding temporal window, the network rapidly mapped the unmodeled disturbance torques to spatial mass asymmetry without requiring persistent excitation maneuvers as illustrated in Fig. 4, the body experienced an immediate lateral drift along the positive X-axis upon takeoff due to the off-center payload. However, designed controller successfully arrested this transient drift and recovered the trajectory. This rapid stabilization was enabled by the PINN periodically converging on the offset parameters (Figs. 5 and 6) and feeding them back into the control allocation matrix. While this represents a partial parameter convergence, it proved entirely sufficient to restore system stability. By periodically identifying and compensating for the majority of the disturbance torque, the updated allocation matrix effectively restored the baseline control, allowing it to reject the remaining unmodeled dynamics without saturating the rotors.

Furthermore, the network successfully converged upon the payload mass prediction. However, as observed in Fig. (2), the mass estimation exhibited cyclical oscillations after the 20-second mark. These periodic fluctuations correspond di-

rectly to the dynamic tracking maneuvers, indicating that the network's mass estimation remains coupled to the multi-agent system's changing inertial movements during maneuvers.

TABLE III
PERFORMANCE METRICS ACROSS MULTIPLE EPISODES

Quality	Ep. 1	Ep. 5	Ep. 10
RMSE Total Loss	0.5462	1.5972	2.2172
RMSE Data Loss	0.0023	0.0017	0.0176
RMSE Physics Loss	0.3155	1.4201	2.042
Offset X RMSE (meters)	0.0153	0.0478	0.0518
Offset Y RMSE (meters)	0.0470	0.0762	0.0543
Trajectory RMSE (meters)	0.4129	0.3757	0.3643
Trajectory x-axis RMSE (meters)	0.2443	0.1521	0.1198
Trajectory y-axis RMSE (meters)	0.2799	0.2946	0.2954
Trajectory z-axis RMSE (meters)	0.1802	0.1768	0.1763
Avg PINN Compute Time (milliseconds)	37.59	34.96	37.18
Avg PINN Compute Time (Hz)	26.6	28.6	26.9

TABLE IV
2nd EPISODE RESULTS

Quality	Value	Unit
RMSE Total Loss	0.6862	-
RMSE Data Loss	0.0016	-
RMSE Physics Loss	0.5230	-
Offset X RMSE	0.1564	meters
Offset Y RMSE	0.0901	meters
Trajectory Loss RMSE	0.4129	meters
Trajectory x-axis RMSE	0.2443	meters
Trajectory y-axis RMSE	0.2799	meters
Trajectory z-axis RMSE	0.1802	meters
Avg PINN Compute Time	37.59	ms
Avg PINN Compute Time	26.6	Hz

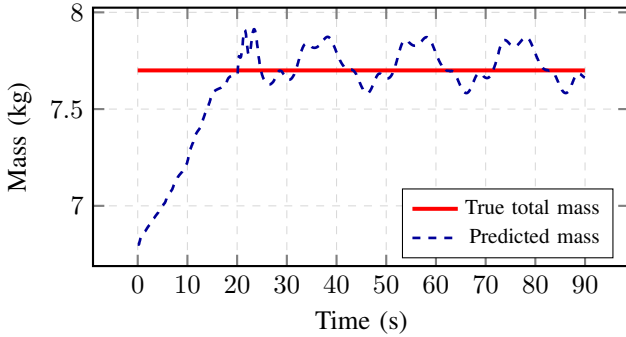


Fig. 2. Mass prediction over-time at 2st episode. The prediction oscillates in a repeated manner following the figure 8 trajectory.

VI. CONCLUSION AND FUTURE WORK

This paper presented a parameter estimation framework for rigid multi-quadrotor systems utilizing an in-the-loop Physics-Informed Neural Network. By formulating a loss function that incorporates Euler-Lagrange and Euler-Poincaré equations directly on the $SO(3)$ manifold, the network guarantees globally valid rigid-body constraints. Furthermore, the optimization is reached just using a short-term receding horizon.

Simulation results demonstrate that integrating this estimator directly into the control allocation matrix allows the

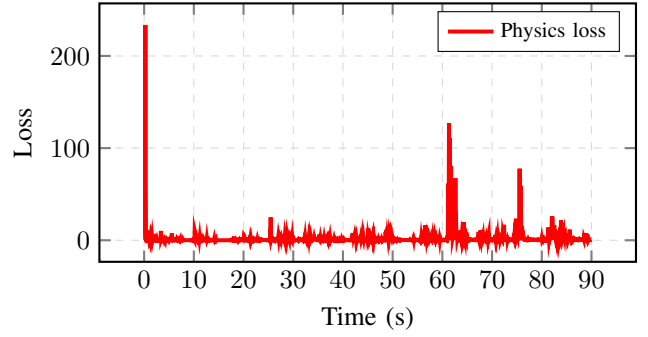


Fig. 3. Physics loss over-time at 2st episode. The PINN is able to arrest the sudden loss spikes and the simulation is done without crashing.

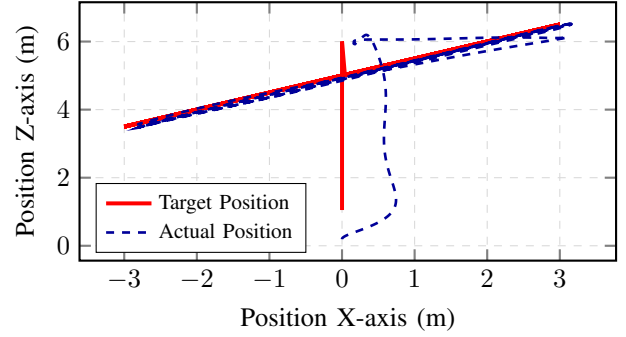


Fig. 4. Body frame position on XZ-axis at 2st episode. The body immediately slides due to the off-center payload. The PINN is able to estimate the offset quickly before it reached the distance error limit.

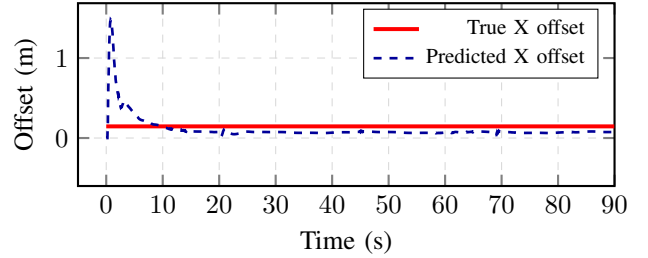


Fig. 5. Predicted payload offset over time along the X-axis during the 2nd episode. The network's ability to rapidly converge upon the true offset during the initial takeoff transient demonstrates converging parameter estimation without the requirement for persistent excitation.

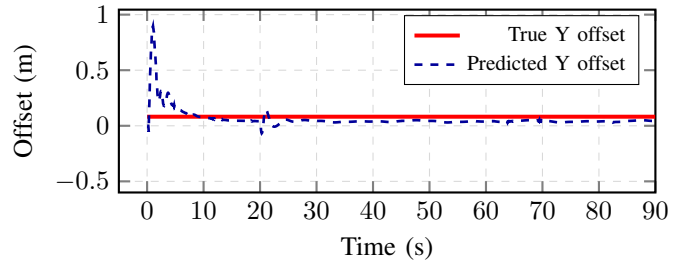


Fig. 6. Predicted payload offset over-time on Y-axis at 2nd episode. The network's ability to rapidly converge upon the true offset during the initial takeoff transient demonstrates converging parameter estimation without the requirement for persistent excitation.

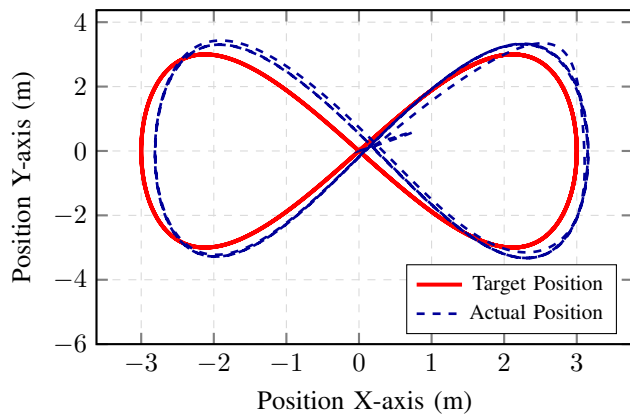


Fig. 7. Target position and actual position at 2nd episode. Even with a sudden drift during takeoff, the controller is able to stabilize and follow the figure 8 trajectory.

system to rapidly identify unmodeled payload mass and the off-center payload shift, arrest transient lateral drift, and maintain stable trajectory tracking without requiring persistent excitation and achieved within the first episode. Even with partial parameter convergence, the dynamic thrust reallocation effectively restores the baseline controller's authority to reject unmodeled disturbance torques. Future research will extend this methodology to dynamic underactuated slung loads, incorporating coupled pendulum dynamics into the PINN's physical residuals to actively dampen payload oscillations.

REFERENCES

- [1] R. Guebsi, S. Mami, and K. Chokmani, "Drones in Precision Agriculture: A Comprehensive Review of Applications, Technologies, and Challenges," *Drones*, vol. 8, no. 11, p. 686, Nov. 2024. [Online]. Available: <https://www.mdpi.com/2504-446X/8/11/686>
- [2] Y. H. Tan, S. Lai, K. Wang, and B. M. Chen, "Cooperative Heavy Lifting Using Unmanned Multi-Agent Systems," in *2018 IEEE 14th International Conference on Control and Automation (ICCA)*. Anchorage, AK: IEEE, Jun. 2018, pp. 1119–1126. [Online]. Available: <https://ieeexplore.ieee.org/document/8444215/>
- [3] J. Zeng, A. M. Gimenez, E. Vinitsky, J. Alonso-Mora, and S. Sun, "Decentralized Aerial Manipulation of a Cable-Suspended Load using Multi-Agent Reinforcement Learning," Nov. 2025, arXiv:2508.01522 [cs]. [Online]. Available: <http://arxiv.org/abs/2508.01522>
- [4] B. Peng, B. Su, K. Yi, H. Wu, and D. He, "Modeling and Analysis of Four-UAV-Slung-Load System," in *2023 35th Chinese Control and Decision Conference (CCDC)*. Yichang, China: IEEE, May 2023, pp. 4289–4294. [Online]. Available: <https://ieeexplore.ieee.org/document/10327396/>
- [5] D. Mellinger, M. Shomin, N. Michael, and V. Kumar, "Cooperative grasping and transport using multiple quadrotors," in *Distributed Autonomous Robotic Systems*. Springer, 2013, pp. 545–558.
- [6] H. Ye, H. Yu, B. Liu, J. Han, Y. Fang, and X. Liang, "Nonlinear Robust Control for Dual-UAV Collaborative Aerial Transportation System," in *2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT)*. Jiaxing, China: IEEE, Nov. 2023, pp. 870–875. [Online]. Available: <https://ieeexplore.ieee.org/document/10528496/>
- [7] C.-A. Lee and T.-H. Cheng, "Concurrent Learning for Cooperative UAV Transportation of Unknown Payloads," *IEEE Open Journal of Control Systems*, vol. 4, pp. 187–198, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/10797683/>
- [8] U. Farid, B. Khan, S. M. Ali, and Z. Ullah, "A Digital Twin Model for UAV Control to Lift Irregular-Shaped Payloads Using Robust Model Predictive Control," *Machines*, vol. 13, no. 11, p. 1069, Nov. 2025. [Online]. Available: <https://www.mdpi.com/2075-1702/13/11/1069>
- [9] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational physics*, vol. 378, pp. 686–707, 2019.
- [10] R. Mojtani, M. Balajewicz, and P. Hassanzadeh, "Kolmogorov n-width and Lagrangian physics-informed neural networks: A causality-conforming manifold for convection-dominated PDEs," *Computer Methods in Applied Mechanics and Engineering*, vol. 404, p. 115810, Feb. 2023. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0045782522007666>
- [11] S. Lakshminarayana, S. Sthapit, and C. Maple, "Application of Physics-Informed Machine Learning Techniques for Power Grid Parameter Estimation," *Sustainability*, vol. 14, no. 4, p. 2051, Feb. 2022. [Online]. Available: <https://www.mdpi.com/2071-1050/14/4/2051>
- [12] M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, Feb. 2019. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0021999118307125>
- [13] G. Serrano, M. Jacinto, J. Ribeiro-Gomes, J. Pinto, B. J. Guerreiro, A. Bernardino, and R. Cunha, "Physics-Informed Neural Network for Multirotor Slung Load Systems Modeling," May 2024, arXiv:2405.09428 [cs]. [Online]. Available: <http://arxiv.org/abs/2405.09428>
- [14] G. Haghighatdoost, "Euler Poincare Dynamics and Control on Lie Groupoids," May 2026, arXiv:2509.20044 [math.DS]. [Online]. Available: <http://arxiv.org/abs/2509.20044>